Tietokantojen perusteet – Kevät 2026

Mikko Egor Legezin

https://github.com/EeegorL/hy_tikape_projektit

Sisältö

1. Tietokannan käyttö 2
2. Tietokannan suunnittelu 5
Käyttäjä voi kirjautua järjestelmään antamalla tunnuksen ja salasanan. 5
Käyttäjä pystyy katsomaan muiden käyttäjien lähettämiä ilmoituksia sekä lähettämään omia ilmoituksia. 6
Ilmoituksessa näkyy myytävän tavaran tiedot (otsikko, kuvaus, hinta) sekä yksi tai useampia kuvia. 6
Ilmoituksessa voi olla luokittelu myytävälle tavaralle (esimerkiksi osasto kirjat, sijainti Helsinki). 6
Käyttäjä voi merkitä ilmoituksen suosikikseen. 6
Kaksi käyttäjää pystyy viestittelemään keskenään ilmoitukseen liittyen. 7
Käyttäjä pystyy merkitsemään tavaran varatuksi ja myydyksi. 7
Tavaran myynnin jälkeen myyjä ja ostaja pystyvät antamaan toisilleen palautteen. 7
CREATE TABLE -lauseet: 8
Users 8
Listings 8
Conversations 8
Messages 8
Images 9
Favorites 9
Keywords 9
Categories 9
Feedback 9
3. Transaktiot 10
Testi 1 10
Testi 2 11
Testi 3 12
Testi 4 13
Testi 5 14
4. Indeksin tehokkuus 15
5. Relaatioalgebra 18
6. NOSQL-tietokanta 19
7. Tietokannan muunnos 20
Lähteet 22

1. Tietokannan käyttö

Luokka Bikes, joka käsittelee bikes_2024.db -tiedoston tietokantaa SQLite-yhteydellä.

```
import sqlite3;

class Bikes:
    def __init__(self):
        self.db = sqlite3.connect("bikes_2024.db");

    def trips_from_station(self, station_name):
        query = f"""
            SELECT COUNT(T.id) FROM Trips T
            JOIN Stations S ON S.id = T.start_station_id
            WHERE S.name = ?
        """;
        return self.db.execute(query, [station_name]).fetchone()[0];

    def trips_to_station(self, station_name):
        query = f"""
            SELECT COUNT(T.id) FROM Trips T
            JOIN Stations S ON S.id = T.end_station_id
            WHERE S.name = ?
        """;
        return self.db.execute(query, [station_name]).fetchone()[0];

    def longest_distance_trips(self, limit):
        query = f"""
            SELECT A.name, B.name, T.distance FROM Trips T
            INNER JOIN Stations A ON A.id = T.start_station_id
            INNER JOIN Stations B ON B.id = T.end_station_id
            GROUP BY T.id
            ORDER BY T.distance DESC
            LIMIT ?;
        """;
        return self.db.execute(query, [limit]).fetchall();

    def longest_duration_trips(self, limit):
        query = f"""
            SELECT A.name, B.name, T.duration FROM Trips T
            INNER JOIN Stations A ON A.id = T.start_station_id
            INNER JOIN Stations B ON B.id = T.end_station_id
            ORDER BY T.duration DESC
            LIMIT ?;
```

```
        """;
        return self.db.execute(query, [limit]).fetchall();

    def trips_during_month(self, month):
        months = {
            "01": "Tammikuu",
            "02": "Helmikuu",
            "03": "Maaliskuu",
            "04": "Huhtikuu",
            "05": "Toukokuu",
            "06": "Kesäkuu",
            "07": "Heinäkuu",
            "08": "Elokuu",
            "09": "Syyskuu",
            "10": "Lokakuu",
            "11": "Marraskuu",
            "12": "Joulukuu"
            };
        month = str(month) if month > 9 else f"0{month}";

        query = f"""
            SELECT COUNT(*) FROM Trips T
            WHERE T.start_time LIKE ?
        """;
        result = self.db.execute(query, [f"%-{month}-%"]).fetchone();
        return (months[month], result[0]);

    def trips_during_weekday(self, day): # https://sqlite.org/lang_datefunc.html
        if(day == 7): day = 0; # "Tässä 1 = maanantai, 2 = tiistai, jne." ei ole kovin kuvaava, sillä ei
selviä onko sunnuntai 0 vai 7. Oletetaan, että käyttäjä syöttäisi 7:n
        days = {
            1: "Maanantai",
            2: "Tiistai",
            3: "Keskiviikko",
            4: "Torstai",
            5: "Perjantai",
            6: "Lauantai",
            0: "Sunnuntai"
        };

        query = """
            SELECT COUNT(*) FROM Trips T
            WHERE STRFTIME('%w', T.start_time) = ?;
        """;
        result = self.db.execute(query, [str(day)]).fetchone();
        return (days[day], result[0])

    def most_popular_start_stations(self, limit):
        query = """
            SELECT S.name, COUNT(T.id) AS x FROM Stations S
            INNER JOIN Trips T ON S.id = T.start_station_id
            GROUP BY S.id
            ORDER BY x DESC
            LIMIT ?;
```

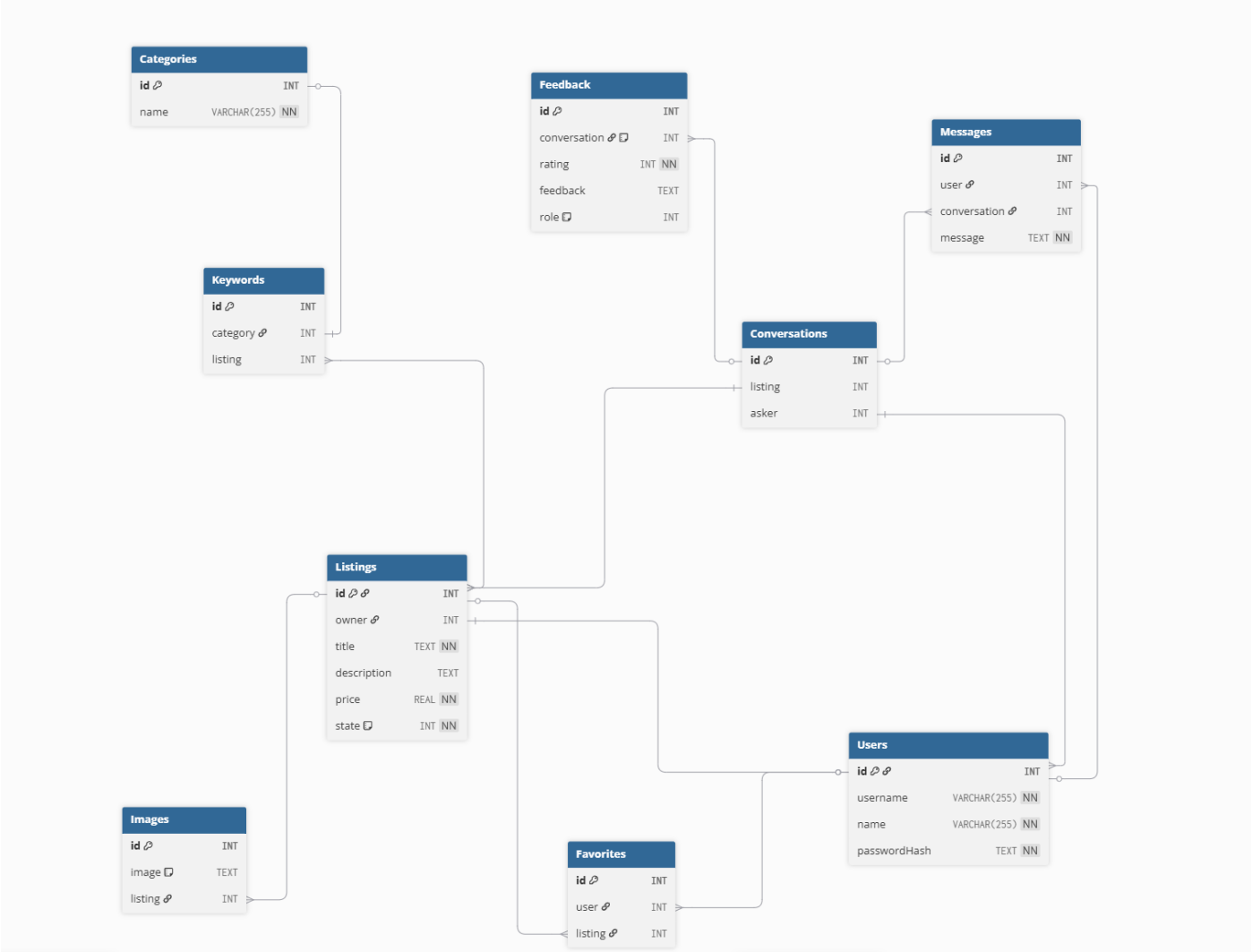
```
        """;
        return self.db.execute(query, [limit]).fetchall();

    def least_popular_start_stations(self, limit):
        query = """
            SELECT S.name, COUNT(T.id) AS n FROM Stations S
            INNER JOIN Trips T ON S.id = T.start_station_id
            GROUP BY S.id
            ORDER BY n ASC
            LIMIT ?;
        """;
        return self.db.execute(query, [limit]).fetchall();

    def largest_differences(self, limit):
        # alussa IFNULLit siltä varalta, että jollakin asemalla ei ole ollut minkäänlaista liikettä
(epätodennäköistä), niin null muuttuu nollaksi
        query = """
            SELECT S.name, IFNULL(A.c, 0), IFNULL(B.c, 0), ABS(IFNULL(A.c, 0) - IFNULL(B.c, 0))) AS diff
            FROM Stations S
            LEFT JOIN (
                SELECT start_station_id, COUNT(*) AS c FROM Trips
                GROUP BY start_station_id
            ) A ON A.start_station_id = S.id
            LEFT JOIN (
                SELECT end_station_id, COUNT(*) AS c FROM Trips
                GROUP BY end_station_id
            ) B ON B.end_station_id = S.id
            ORDER BY diff DESC
            LIMIT ?;
        """;
        return self.db.execute(query, [limit]).fetchall();
```

2. Tietokannan suunnittelu

Myyntikauppapaikan kaltaisen verkkopalvelun SQL-skeema, tauluineen ja selityksineen:



Muutos- ja lisäysoperaatioissa oletetaan, että käyttäjä on kirjautunut järjestelmään, ja siten valtuutettu tekemään muutoksia. Syntyisi turhaa toistoa, jos asia mainittaisiin aina erikseen.

Esim. tekstissä "Myyjä voi lisätä ilmoitu*kseensa* asiasanoja..." oletetaan, että myyjä on tunnistettu ilmoituksen omistajaksi kirjautumistiedoista.

Käyttäjä voi kirjautua järjestelmään antamalla tunnuksen ja salasanan.

Käyttäjä kirjautuu järjestelmään tauluun **Users** tallennetun käyttäjänimeä (*username*) ja salasanaa vastaavaa hashia (*passwordHash*) käyttämällä.

Käyttäjä pystyy katsomaan muiden käyttäjien lähettämiä ilmoituksia sekä lähettämään omia ilmoituksia.

Järjestelmä hakee käyttäjälle näytettäväksi taulun **Listings** mukaiset ilmoitukset.

Ilmoituksesta ilmenee sen tekijä. Tietokantatasolla yksittäinen *listing* viittaa taulun **Users** *id*-kenttään oman kenttänsä *owner* arvolla.

Käyttäjä voi myös itse luoda ilmoituksia, jolloin *listingin owner*-kenttään tulee kyseisen käyttäjän *id*.

Ilmoituksessa näkyy myytävän tavaran tiedot (otsikko, kuvaus, hinta) sekä yksi tai useampia kuvia.

Ilmoitus sisältää niin tekstimuotoista tietoa, kuten otsikon ja kuvauksen jotka tallentuvat **Listings**-taulun, sekä tarvittaessa kuvia, jotka tallentuvat tauluun **Images**, linkittyen asianmukaiseen *listingiin* sen *id*:n perusteella oman *listing*-kentän arvolla.

Ilmoituksessa voi olla luokittelu myytävälle tavaralle (esimerkiksi osasto kirjat, sijainti Helsinki).

Myyjä voi lisätä ilmoitukseensa asiasanoja parantaakseen ilmoituksensa näkyvyyttä asiakkaan suodattaessa tuloksia.

Asiasanavaihtoehdot ovat määritelty taulussa **Categories**, sisältäen kategorian nimen.

Myyjän lisättyä ilmoitukselleen asiasanan, tämä tallentuu "välikäsitauluun" **Keywords**, joka sisältää *listing*-viiteavaimen kyseisen *listingin id*:hen, sekä *category*-viiteavaimen, joka viittaa **Categories**-taulun kategorioihin.

Käyttäjä voi merkitä ilmoituksen suosikikseen.

Käyttäjä voi merkitä ilmoituksen suosikikseen, jolloin tieto tästä tallentuu tauluun **Favorites**, joka sisältää **Listings**-taulun *id*:hen viittaavan viiteavaimen *listing* ilmoituksesta jota tietue koskee, ja viiteavaimen *user*, joka viittaa ilmoituksen suosikiksi lisänneen käyttäjän **Users**-taulun mukaiseen *id*:hen.

Tällöin voidaan hakea vain ne ilmoitukset, joiden id löytyy Favorites-taulusta.

Kaksi käyttäjää pystyy viestittelemään keskenään ilmoitukseen liittyen.

Kaksi käyttäjää voivat keskustella ilmoituksen suhteen, jolloin keskustelu itse tallentuu tauluun **Conversations**, ja keskustelun viestit tauluun **Messages**.

Nämä taulut linkittyvät **Messages**-taulun viiteavaimen *conversation* viitaten **Conversations**-taulun *id*-kenttään.

Conversations-taulu itsessään sisältää osapuolista tiedon vain kysyjästä/ostajasta kentällä *asker*, viitaten taulun **Users** *id*:hen. Myyjän tiedot voidaan selvittää kyselyllä **Conversations**-taulun viiteavaimella *listing*, joka viittaa *listingin id*:hen. **Listing**-taulu puolestaan sisältää tiedon sen luojasta.

Taulu **Messages** sisältää tiedon sen lähettäneestä käyttäjästä *id*:n muodossa kentässä *user*, joka viittaa tauluun **User**, tiedon keskustelusta johon se liittyy, avaimen *conversation*-muodossa, sekä itse lähetetyn viestin tekstisisällön.

Käyttäjät eivät pysty jakamaan toisilleen kuvia tms. tietomuotoja, ainoastaan tekstiä.

Käyttäjä pystyy merkitsemään tavaran varatuksi ja myydyksi.

Myyjä voi vaihtaa listauksensa tilan oletuksesta joko varatuksi tai myydyksi, joka näkyy muille käyttäjille.

Myyjän päivitys tallentuu **Listings**-taulun tietueen *state*-kenttään, jolla on ilmoituksen tilaa kuvaava numeerinen arvo, esim. 0 = myynnissä, 1 = myyty ja 2 = varattu.

Tavaran myynnin jälkeen myyjä ja ostaja pystyvät antamaan toisilleen palautteen.

Kun tavara on myyty, eli *listingin* kentän state arvo on edellisen kohdan mukaisesti 1, myyjälle ja ostajalle avautuu mahdollisuus antaa toisilleen palautetta.

Palaute tallentuu tauluun **Feedback**, joka linkittyy kentän *conversation*-viitteen avulla taulun **Conversations**-keskusteluun.

Taulussa käyttäjältä pyydetään perinteinen 1-5 arviointi ja mahdollisuus tekstimuotoiseen palautteeseen.

Taulussa on myös kenttä *role*, joka määrää missä roolissa arvosteleva käyttäjä on, myyjä vai ostaja. Näin, käyttäjällä voi olla esim. mainio maine myyjänä, mutta surkea maine ostajana, jos arvostelut ovat julkisia.

CREATE TABLE -lauseet:

Users

```
CREATE TABLE Users(
  id INT PRIMARY KEY,
  username VARCHAR(255) NOT NULL,
  name VARCHAR(255) NOT NULL,
  passwordHash TEXT NOT NULL
);
```

Listings

```
CREATE TABLE Listings(
  id INT PRIMARY KEY,
  owner INT REFERENCES Users(id),
  title TEXT NOT NULL,
  description TEXT,
  price REAL NOT NULL,
  state INT NOT NULL
);
```

Conversations

```
CREATE TABLE Conversations(
  id INT PRIMARY KEY,
  listing INT REFERENCES Listings(id),
  asker INT REFERENCES User(id)
);
```

Messages

```
CREATE TABLE Messages(
  id INT PRIMARY KEY,
  user INT REFERENCES User(id),
  conversation INT REFERENCES Conversations(id),
  message TEXT NOT NULL
);
```

Images

```
CREATE TABLE Images(
  id INT PRIMARY KEY,
  image TEXT,
  listing INT REFERENCES Listings(id)
);
```

Favorites

```
CREATE TABLE Favorites(
  id INT PRIMARY KEY,
  user INT REFERENCES Users(id),
  listing INT REFERENCES Listings(id)
);
```

Keywords

```
CREATE TABLE Keywords(
  id INT PRIMARY KEY,
  category INT REFERENCES Categories(id),
  listing INT REFERENCES Listings(id)
);
```

Categories

```
CREATE TABLE Categories(
  id INT PRIMARY KEY,
  name VARCHAR(255) NOT NULL
  /* description TEXT NOT NULL */ /* the category could have a description, but probably not necessary */
);
```

Feedback

```
CREATE TABLE Feedback(
  id INT PRIMARY KEY,
  conversation INT REFERENCES Conversations(id), /* feedback is linked to the conversation between buyer and seller rather than the listing */
  rating INT NOT NULL, /* e.g. 1-5 */
  feedback TEXT
);
```

3. Transaktiot

Kuvatkoon K1 ja K2 kahta erillistä konsolia / käyttäjäistuntoa

Taulu **Tables** olkoon seuraavanlainen:

```
CREATE TABLE Test (x INTEGER);

INSERT INTO Test (x) VALUES (1);
```

Testi 1

```
K1: BEGIN;

K1: SELECT x FROM Test;



K2: BEGIN;

K2: UPDATE Test SET x = 2;

K2: SELECT x FROM Test;

K2: COMMIT;



K1: COMMIT;
```

Testi menee läpi.

1. SELECT-lause palauttaa arvon 1

2. SELECT-lause palauttaa arvon 2

Molemmat transaktiot onnistuvat, koska toisin kuin esim päivitys- tai lisäysoperaatiossa, lukuoperaatio ei lukitse tietokantaa transaktion ajaksi.

Testi 2

```
K1: BEGIN;

K1: SELECT x FROM Test;

K1: UPDATE Test SET x = 2;



K2: BEGIN;

K2: SELECT x FROM Test;

K2: COMMIT;



K1: COMMIT;
```

Testi menee läpi.

1. SELECT-lause palauttaa arvon 1

2. SELECT-lause palauttaa arvon 2

Molemmat transaktiot onnistuvat. Toinen SELECT-lause kuitenkin eroaa ensimmäisen testin tuloksesta, koska toisessa transaktiossa oleva SELECT-lause ei näe muiden transaktioiden "luonnosarvoja" ennen kuin ne commitataan.

Testi 3

```
K1: BEGIN;

K1: UPDATE Test SET x = 2;


K2: BEGIN;

K2: UPDATE Test SET x = 3;

K2: SELECT x FROM Test;

K2: COMMIT;


K1: SELECT x FROM Test;

K1: COMMIT;
```

Testi ei mene läpi.

1. SELECT-lause palauttaa arvon 1
2. SELECT-lause palauttaa arvon 2

Vain ensimmäinen transaktio onnistuu. Toinen transaktio (K2) kaatuu, sillä K1:n transaktion UPDATE-lause lukitsee tietokannan käyttöönsä estäen sen muokkaamisen siihen asti, kunnes tämä (K1) transaktio committaa (tai peruu) muutoksensa.

Testi 4

```
K1: BEGIN;

K1: UPDATE Test SET x = 2;



K2: BEGIN;

K2: UPDATE Test SET x = 2;

K2: SELECT x FROM Test;

K2: COMMIT;



K1: SELECT x FROM Test;

K1: COMMIT;
```

Testi ei mene läpi.

1. SELECT-lause palauttaa 1
2. SELECT-lause palauttaa 2

Vain K1-transaktio onnistuu, sillä K2 yrittää päivittää tietokannan tilaa sen ollen lukittuna keskenolevan K1-transaktion vuoksi.

Testi 5

```
K1: BEGIN;
K1: SELECT x FROM Test;


K2: BEGIN;
K2: SELECT x FROM Test;
K2: UPDATE Test SET x = 2;
K2: COMMIT;


K1: UPDATE Test SET x = 2;
K1: COMMIT;
```

Testi ei mene läpi.

1. SELECT-lause palauttaa 1
2. SELECT-lause palauttaa 1

Kumpikin transaktio epäonnistuu.

Etenemisjärjestyksessä, K2 transaktio epäonnistuu, koska se ei saa *commitoitua* muutostaan tietokannan ollen myös toisen transaktion käytössä. Transaktion UPDATE-lause kyllä menee läpi, mutta sitä ei saada "vahvistettua".

K1 transaktio ei puolestaan onnistu, koska tietokanta on lukittu muutoksilta K2-transaktiolla. Syntyy tällainen noidankehä.

Eteenpäin päästään vasta kun K1-transaktio viedään loppuun commitilla (joka ei "muutoslukon" vuoksi voi tehdä muuta kuin committaa ilman muutoksia) jonka jälkeen K2 voi tehdä mitä lystää vapautuneella tietokannalla.

4. Indeksin tehokkuus

Testaamiseen käytetty koodi, testit omissa funktioissaan:

```
import sqlite3, random, string, os;
from time import time;

drop = "DROP TABLE IF EXISTS Movies;";
create = "CREATE TABLE Movies(id INT PRIMARY KEY, name VARCHAR(255), release_year INT);";
insert = "INSERT INTO Movies VALUES(?, ?, ?);";
select = "SELECT COUNT(*) FROM Movies WHERE release_year = ?;";

def test1():
    db = sqlite3.connect("t1.sql");
    db.execute("BEGIN;");
    db.execute(drop);

    db.execute(create);

    rowAddStart = time();
    for i in range(0, 10**6):
        randName = "".join([random.choice(string.ascii_lowercase) for i in range(0, 8)]);
        randYear = random.randint(1900, 2000);
        db.execute(insert, [i, randName, randYear]);
    rowAddEnd = time();

    queryStart = time();
    for i in range(0, 10**3):
        randYear = random.randint(1900, 2000);
        db.execute(select, [randYear]);
    queryEnd = time();

    fileSize = os.path.getsize("./t1.sql");
    db.execute("COMMIT;");
    return (
        f'{round(rowAddEnd - rowAddStart)}s',
        f'{round(queryEnd - queryStart)}s',
        f'{round(fileSize / 10**6)} Mb'
    );

def test2():
    db = sqlite3.connect("t2.sql");
    db.execute("BEGIN;");
    index = "CREATE INDEX year ON Movies(release_year);";
    db.execute(drop);

    db.execute(create);
    db.execute(index);

    rowAddStart = time();
    for i in range(0, 10**6):
        randName = "".join([random.choice(string.ascii_lowercase) for i in range(0, 8)]);
        randYear = random.randint(1900, 2000);
        db.execute(insert, [i, randName, randYear]);
```

```
    rowAddEnd = time();

    queryStart = time();
    for i in range(0, 10**3):
        randYear = random.randint(1900, 2000);
        db.execute(select, [randYear]);
    queryEnd = time();

    fileSize = os.path.getsize("./t2.sql");
    db.execute("COMMIT;");
    return (
        f'{round(rowAddEnd - rowAddStart)}s',
        f'{round(queryEnd - queryStart)}s',
        f'{round(fileSize / 10**6)} Mb'
    );

def test3():
    db = sqlite3.connect("t3.sql");
    db.execute("BEGIN;");
    index = "CREATE INDEX year ON Movies(release_year);";
    db.execute(drop);

    db.execute(create);

    rowAddStart = time();
    for i in range(0, 10**6):
        randName = "".join([random.choice(string.ascii_lowercase) for i in range(0, 8)]);
        randYear = random.randint(1900, 2000);
        db.execute(insert, [i, randName, randYear]);
    rowAddEnd = time();

    db.execute(index);
    queryStart = time();
    for i in range(0, 10**3):
        randYear = random.randint(1900, 2000);
        db.execute(select, [randYear]);
    queryEnd = time();
    db.execute("COMMIT;");
    fileSize = os.path.getsize("./t3.sql");

    return (
        f'{round(rowAddEnd - rowAddStart)}s',
        f'{round(queryEnd - queryStart)}s',
        f'{round(fileSize / 10**6)} Mb'
    );

print(test1());
print(test2());
print(test3());
```

Testi 1:

- Rivien lisääminen: 3 sek

- SELECT-kysely: 35 sek

- Tiedoston koko: 37 Mb

Testi 2:

- Rivien lisääminen: 5 sek

- SELECT-kysely: 0 sek

- Tiedoston koko: 49 Mb

Testi 3:

- Rivien lisääminen: 3 sek

- SELECT-kysely: 0 sek

- Tiedoston koko: 48 Mb

Testi 1 luo tietokannan ilman indeksilisäämisen vaatimaa lisätyötä, ja on siksi nopea. Myös tiedostokoko on testeistä pienin, sillä indeksit eivät ole viemässä tilaa.

Hakuvaiheessa prosessi kuitenkin hidastuu, koska tietokanta joudutaan skannaamaan ja jokainen rivi nuuhkimaan läpi.

Testi 2 on hitain, sillä testi tekee "tuplatyötä", lisäten tauluun itse tietoa ja sitä vastaavia indeksejä, päivittäen indeksejä joka lisäyksellä.

Kun indeksit ovat olemassa, haku kuitenkin nopeutuu merkittävästi (vrt. 35s vs 0s), kun tietokanta "tietää" missä halutut arvot ovat indeksien perusteella.

Testi 3 on kuitenkin voittaja, joka on kaikilta osin nopea.

Se eroaa ensimmäisestä testistä indeksillään, ja toisesta sillä, että indeksi luodaan vasta kun kaikki rivit ovat lisätty tauluun. Tämä aiheuttaa sen, että indeksit luodaan vain kerran, eikä niitä tarvitse päivittää jokaisen lisäyksen yhteydessä kuin edellisessä testissä.

5. Relaatioalgebra

```
class Relation:
    def __init__(self, fields):
        self.fields = fields;
        self.rows = [];

    def __str__(self):
        return str({tuple(x.values()) for x in self.rows});

    def add_tuple(self, tup):
        if(len(self.fields)) == 1:
            self.rows.append({self.fields: tup});
        else:
            self.rows.append({self.fields[i]: tup[i] for i in range(0, len(self.fields))})

def projection(dict, keysTup):
    if(isinstance(dict, Relation)): dict = dict.rows; # is an instance of Relation-class
    return {tuple([row[key] for key in row.keys() if(key in keysTup)]) for row in dict};

def restriction(dict, key, val):
    if(isinstance(dict, Relation)): dict = dict.rows; # is an instance of Relation-class
    return tuple([x for x in dict if(x[key] == val)]);

if(__name__ == "__main__"):
    products = Relation(("id", "name", "price"))

    products.add_tuple((1, "retiisi", 7))
    products.add_tuple((2, "porkkana", 5))
    products.add_tuple((3, "nauris", 4))
    products.add_tuple((4, "lanttu", 8))
    products.add_tuple((5, "selleri", 4))

    print(products)

    print(projection(products, ("name")))
    print(projection(products, ("price")))
    print(projection(products, ("name", "price")))

    print(restriction(products, "name", "porkkana"))
    print(restriction(products, "price", 4))

    print(projection(restriction(products, "price", 4), ("name")))
```

6. NOSQL-tietokanta

```
import pymongo;

connection_string = "mongodb+srv://tikape:NAq8a4pNLWF8TMfd@cluster0.u4vehy9.mongodb.net/"
client = pymongo.MongoClient(connection_string);

database = client.get_database("tikape");
apartments = database["apartments"];

postinro_00700 = apartments.count_documents({"zip_code": "00700"});
print(f'Monessako asunnossa postinumero on 00700? - {postinro_00700}');

rakennusvuosi_2000 = apartments.count_documents({"construction_year": {"$gte": 2000}});
print(f'Monessako asunnossa rakennusvuosi on 2000-luvulla? - {rakennusvuosi_2000}');

pinta_ala = apartments.count_documents({"apartment_size": {"$gte": 50, "$lte": 70}});
print(f'Monessako asunnossa pinta-ala on välillä 50–70 m²? - {pinta_ala}');

myyty_ainakin_kerran = apartments.count_documents({"transactions.date": {
    "$gte": "2010-01-01",
    "$lte": "2012-12-31"
        }
    }
);
print(f'Moniko asunnoista on myyty ainakin kerran vuosina 2010–2012? - {myyty_ainakin_kerran}');

kallein = apartments.find_one({}, {"transactions": 1}, sort={"transactions.selling_price": -1});
kallein_hinta = max(kallein["transactions"], key=lambda x: x["selling_price"])["selling_price"];
print(f'Mikä on kallein asunnon myyntihinta aineistossa? - {kallein_hinta}');
```

7. Tietokannan muunnos

```
import sqlite3, os;


files = os.listdir("od-trips-2023");

trips = [];
stations = {};

for file in files:
    with open(f'od-trips-2023/{file}', encoding="utf-8") as f:
        next(f); # skip headers
        for line in f:
            data = line.split(",");

            if(not data[4].isnumeric()):
                data[3] = (data[3] + "," + data[4]).replace('"', "");
                del data[4];

            if(not data[6].isnumeric()):
                if(data[6].replace(".","").isdigit()):
                    data[6] = round(float(data[6])); # löytyy floateja integerejen sijaan...
                    continue;
                data[5] = (data[5] + "," + data[6]).replace('"', "");
                del data[6];

                if(data[6] == ""): data[6] = 0; # erikois-erikoistapaus: Aalto + puuttuva data

            if(len(data) == 7): # Virheellisen datan käsittely
                data[5] = data[5][:-1]; # poistaa pilkun virhedatassa
                data.append(data[6]);
                data[6] ="0"; # asettaa puuttuvan Covered distance -arvon nollaksi

            trips.append(
                {
                    "start_time": data[0],
                    "end_time": data[1],
                    "start_station_id": int(data[2]),
                    "end_station_id": int(data[4]),
                    "distance": round(float(data[6])), # joissakin on float-muotoista dataa
                    "duration": round(float(data[7]))
                }
            );

            stations[int(data[2])] = data[3];
            stations[int(data[4])] = data[5];

        f.close();


db = sqlite3.connect("bikes_2023.db");
```

```
db.execute("DROP TABLE IF EXISTS Trips");
db.execute("""CREATE TABLE Trips (
    id INTEGER PRIMARY KEY,
    start_time TEXT,
    end_time TEXT,
    start_station_id INTEGER REFERENCES Stations,
    end_station_id INTEGER REFERENCES Stations,
    distance INTEGER,
    duration INTEGER
);"""");

db.execute("DROP TABLE IF EXISTS Stations");
db.execute("""CREATE TABLE Stations (
    id INTEGER PRIMARY KEY,
    name TEXT
);"""");

# Stations
db.execute("BEGIN;");
for x in stations:
    db.execute("INSERT INTO Stations(id, name) VALUES(?, ?)", [x, stations[x]]);
db.execute("COMMIT;");

# Trips
db.execute("BEGIN;");
for x in trips:
    data = [x for x in x.values()];
    db.execute("INSERT INTO Trips(start_time, end_time, start_station_id, end_station_id, distance,
duration) VALUES(?, ?, ?, ?, ?, ?);", data);
db.execute("COMMIT;");

if(__name__ == "__main__"):
    print(db.execute("SELECT COUNT(*) FROM Stations;").fetchone());
    print(db.execute("SELECT COUNT(*) FROM Trips;").fetchone());
    print(db.execute("SELECT * FROM Stations WHERE id = 100;").fetchone());
    print(db.execute("SELECT * FROM Trips WHERE id = 123456;").fetchone());
```

Lähteet

Käytin hyväkseni ja silmäilin lukuisia keskusteluja sivuilla ja forumeilla kuten muun muassa:

- www.geeksforgeeks.org
- www.stackoverflow.com
- www.reddit.com

joten en listaa jokaista jota vilkaisin.

Näistä katsoin lähinnä Pythonin syntaksia ja utf-8:n saamista csv:stä, mm.:
https://stackoverflow.com/questions/69399503/how-to-return-the-last-character-of-a-string-in-python

https://www.reddit.com/r/learnpython/comments/5aqovk/python_cant_analyse_the_letters_%C3%A6_%C3%B8_and_%C3%A5/

sekä SQLiten erikoisuuksia esim. primary keyn suhteen:

https://stackoverflow.com/questions/6982173/sqlite-auto-increment-non-primary-key-field

Tekoälyä en erikoisemmin yrittänyt käyttää, mutta jokainen Google-haku luo tekoälykoosteen, niin sitä on vaikeaa paeta.

Googlen tekoälyä siis samankaltaisiin kysymyksiin kuin ylämainitut.

MongoDB oli jo ennestään jokseenkin tuttu, mutta piti tutustua filttereiden toimintaan (gte, lte ym.) ja Mongon arvojen sisällä vertaamiseen:
https://www.w3schools.com/mongodb/mongodb_query_operators.php

https://www.mongodb.com/docs/manual/reference/mql/query-predicates/comparison/#std-label-query-selectors-comparison

https://stackoverflow.com/questions/4791555/how-do-i-do-greater-than-less-than-using-mongodb/4791583
https://www.mongodb.com/docs/manual/tutorial/query-embedded-documents/